

Sensors and Op-amps

Learning Objectives:

- Model op amp comparator mode using MATLAB.
 - Create and use a user-defined function.
 - Plot with subplots.
- Simulate a sensor-comparator circuit in LTSpice, feeding a sinewave to one of its inputs.

Overview: In your labs this week, you will simulate and build a comparator circuit that will turn an LED on or off depending on the resistance of a sensor. The sensor you will use is a photoresistor which changes resistance with light.

Part 1: Simulate an Op Amp Comparator (Switch) using LTSpice: (25 points)

Our comparator circuit compares two voltages, v_p and v_n , and turns ON (outputs 5V) if $v_p > v_n$ or OFF (outputs 0V) if $v_p < v_n$. Our circuit will use an op amp in comparator configuration (referred to as “comparator mode”) to make the voltage comparison. NOTE: The case $v_p = v_n$ is unstable for an op amp in comparator mode, so it is generally avoided in considering the operation of a comparator.

- a) In LTSpice, you will build a full sensor comparator circuit in steps, checking each step to be sure your circuit is working as you expect. Building a circuit in steps is good practice in the lab, as it allows you to debug your circuit as you go.

Step 1 — The Comparator: (Fig. 1.) Start with the op amp positive supply connected to the 5V source (often labeled V_{DD} , V_{CC} , or V_{EE}), and the negative supply connected to GND. Connect the v_p and v_n inputs to two different voltage sources. Change the input voltages manually, and record the output voltage at the op amp V_{out} .

NOTE: The op amp outputs less than V_{CC} due to the loss of energy used to make it function internally.

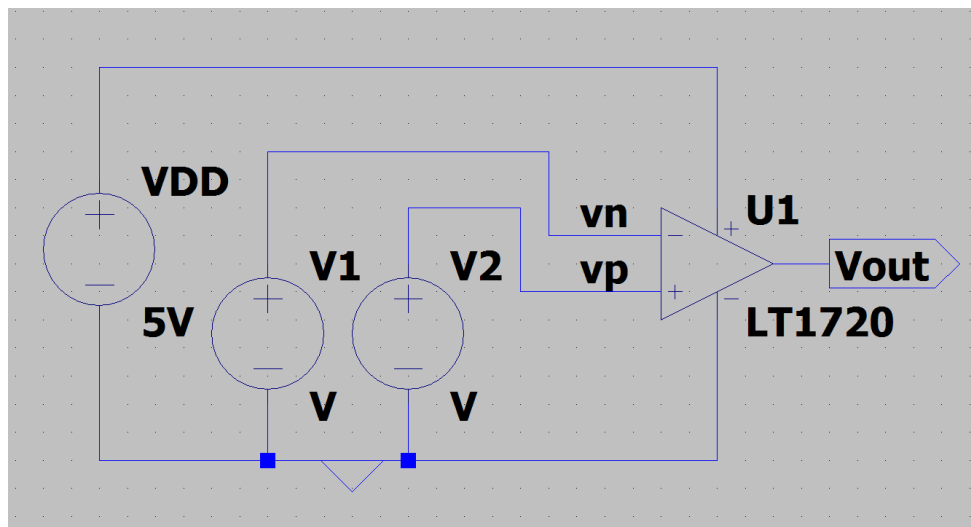


Fig. 1: Step 1 – Comparator op amp circuit.

Fill in the Expected column first in the The Table Below, then confirm the comparator works as you expect using LTspice and fill in the simulated column.

The Table Below

v_p	v_n	V_{out} (expected)	V_{out} (simulated)
5V	4V		
4.1V	4V		
3.9V	4V		
0.1V	4V		
5V	2V		
2.1V	2V		
1.9V	2V		
0.1V	2V		

Step 2 — The sensor voltage divider: (Fig. 2). The v_p comparator input voltage is determined by a voltage divider made from R_{sensor} and R_3 . For the purposes of LTspice simulation, we will use a regular resistor to represent R_{sensor} (a pot is shown as a real-world adjustable resistor).

Choose an ON/OFF-transition resistance value for the sensor in the reduced range of your photosensor between 200Ω – $20k\Omega$ such that your comparator switches ON/OFF at this value. (FYI – ambient light in the MEB 2555 lab room sets R_{sensor} to about $2k\Omega$.)

$$R_{sensor,ON} = \text{_____} \Omega$$

Choose R_3 . A good value for R_3 is $R_{sensor,ON}$. This will allow the voltage divider to be roughly in the middle of its range when the comparator switches ON/OFF. Choose a standard resistor for R_3 that is near this value.

$$R_3 = \text{_____} \Omega$$

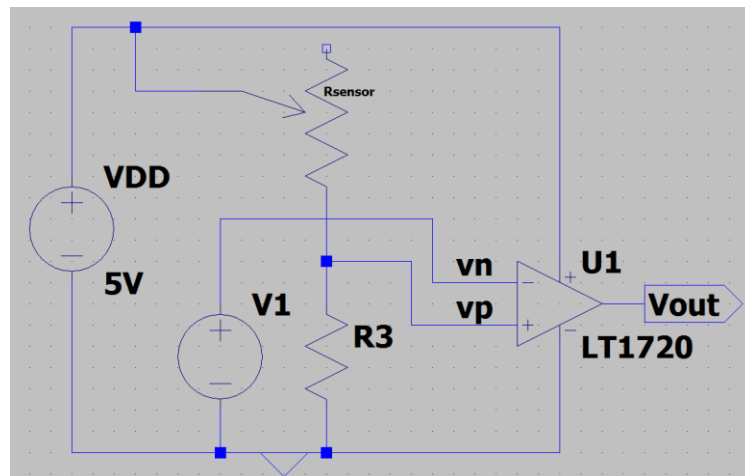


Fig. 2: Step 2 – Comparator op amp circuit with sensor- R_3 v-divider at v_p .

Add the sensor- R_3 voltage divider to your circuit to replace V_2 , as in Fig. 2. The pot is unavailable in the standard LTspice library, so you must download both pot.asc and pot.asy files from canvas and add them to the same folder as the circuit. Use LTspice to determine the value

of v_p that is associated with the resistance values you have chosen. To set R_{sensor} using the bottom resistor in the pot, right-click on the pot and set “val” to a percentage of “R” equal to your target resistance. Set “R” to the value of one of your pots in your kit that is at least $20\text{k}\Omega$.

$v_p \text{ ON} = \underline{\hspace{2cm}} \text{ V}$

Step 3 – The reference voltage. (Fig. 3). The reference (threshold) voltage, v_n , should be set equal to the $v_p \text{ ON}$ voltage you determined above. The reference voltage is also created from a voltage divider. In this case, we use a single pot for both resistors, R_a and R_b , making up the voltage divider as shown in Fig. 3. Choose a pot from your kit that is at least $1\text{k}\Omega$.

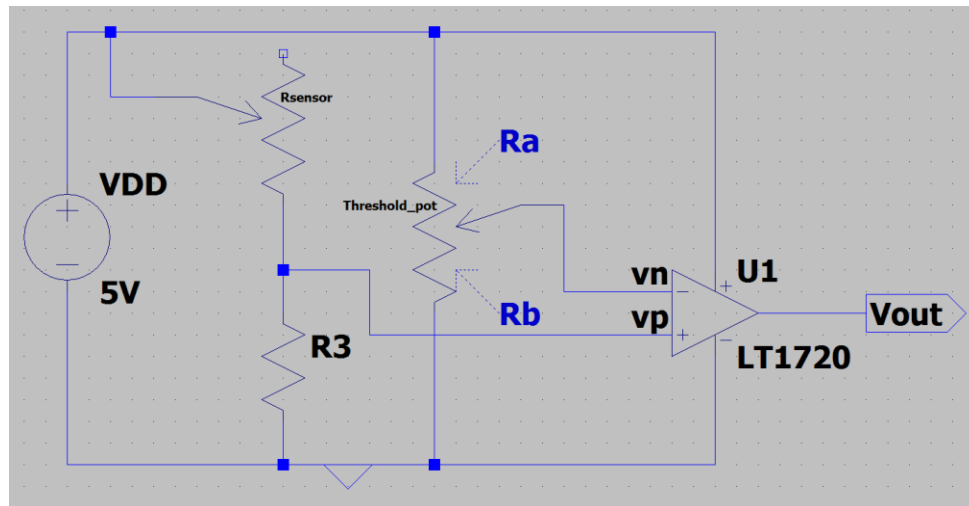


Fig. 3: Step 3 – Comparator op amp circuit with v-divider at v_n added to front-end.

$R_{\text{pot}} = \underline{\hspace{2cm}} \Omega$

What values of R_a and R_b should you tune this pot to in order to obtain the value of v_n you want? It's a trivial calculation, and also a good exercise in using LTspice to sweep through resistance values in a pot, so please do both. The tutorial video linked on the canvas page for this lab walks you through importing and using the pot model (start at 11:09). Sweep through the full range of values of R_{pot} (1-100%) to determine values for R_a and R_b corresponding to your target value for v_n :

$R_a = \underline{\hspace{2cm}} \Omega$ (calculated) $R_b = \underline{\hspace{2cm}} \Omega$ (calculated)

$R_a = \underline{\hspace{2cm}} \Omega$ (simulated) $R_b = \underline{\hspace{2cm}} \Omega$ (simulated)

Step 4 – The LED indicator light: (Fig. 4). Use the LED shown in Fig. 4. To specify this model in LTspice, after placing a generic diode right-click on the diode and select *pick new diode*. This LED is similar to the red one you used in Lab 3, with a 330Ω resistor in series to limit the current through it.

Unfortunately, in LTspice there is no way to know when the LED is ON other than by measuring the LED voltage and current. Similarly to what you have done in step 3, sweep through the full range of resistances of your photosensor ($200\Omega - 20\text{k}\Omega$) with adequate resolution to capture the switching point of your output, and plot both op amp input voltages and

its output voltage on one figure, and then plot the LED voltage drop and current on another figure. See Fig. 5 and Fig. 6 for what to expect in your plots.

NOTE: To measure a voltage drop, as opposed to a node voltage, click-and-drag the red probe from the wire above the LED to the wire on the other side of the LED when the probe has turned black, then release the click.

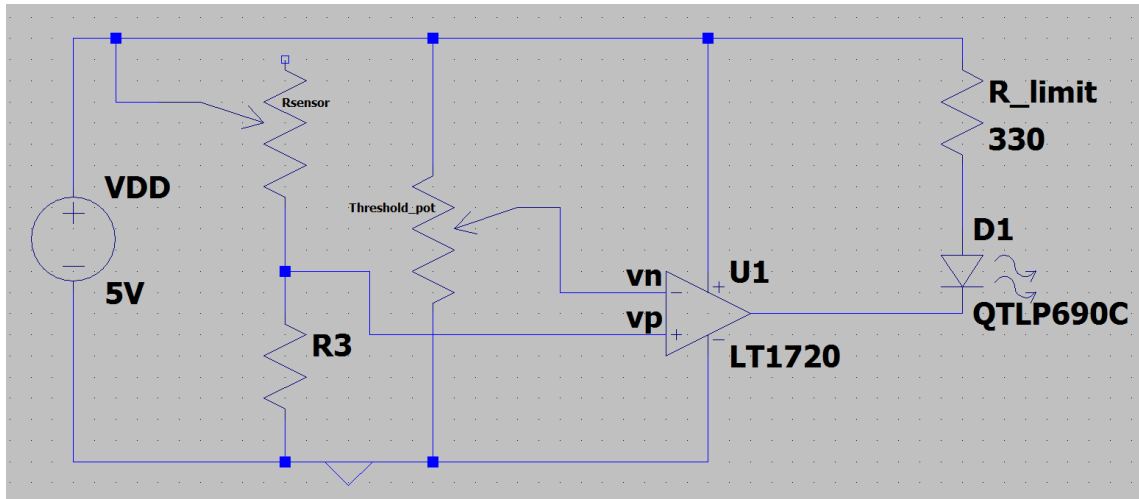


Fig. 4: Step 4 – Comparator op amp full sensor circuit with LED output.



Fig. 5: Input voltages v_n and v_p , and V_{out} .

Fig. 6: V_{out} , V_{LED} and I_{LED} .**Part 2: Simulate a comparator using Matlab: (25 points)**

Now you will implement the same circuit as in Part 1 using Matlab. **Re-write all code using proper variable names and comments that are relevant to the problem.**

- a) Write a voltage-divider equation in Matlab to transform changes in resistance to changes in voltage. Do this only for your sensor-divider at v_p , and set the reference voltage v_n to the same value you determined in Part 1. Note the use of the element-wise dot operator for array division.

```
div_arr = full*(R ./ (R + R_arr)); % example
```

- b) Create a Matlab function called *comparator* that accepts a vector (v_p) and a scalar (v_n) as its two input arguments, and outputs a vector of values (V_{out}) that are 5V whenever $v_p > v_n$ or 0V whenever $v_p \leq v_n$. The example below only finds equality, but you need more than that. Check the Matlab documentation on the equality function used here, and at the bottom of the page you may be led to the exact function for your needs. Also, if needed, see the Matlab Onramp section on how to write functions.
- c) Test your comparator by sweeping R_{sensor} over its full range as given in Part 1.

Here is an example script similar to your task (rename relevant variables, etc.):

```
% Script made to call do_this function
arr = linspace(1,3,100); % enter an array
NUM = 2; % enter a scalar value
% Call the do_this user-defined function, and return its
% output as result_array:
result_array = do_this(arr, NUM);

% Create a user-defined function called do_this
function out_arr = do_this(in_arr, in_num)
% Note that the input and output names do NOT need to be
% the same as the variables used when calling the function
    Out_arr = eq(in_arr, in_num); % check for equality
end
```

In this example, the function *do_this* checks every element in the array named *in_arr* against the single scalar value stored in variable *in_num*, and returns either 1 or 0 in each corresponding array location in *out_arr* depending on whether the elements are equal.

NOTE: When the function is called in the same script as the main program, as in the case above, the function definition must be at the bottom of the script. Alternatively, you could put the main program in one script and the function in another. The name of the script that holds the function must have the same name as the function itself, and must be in the Matlab current directory.

- d) Now, let's try a little fancier test of your *comparator* function. To model the brightness of the 24-hour cycle of the sun on Earth, which follows a sinewave, use an input vector *Rsensor* that models seven full revolutions (one week) around the sun with a sinewave that has a full swing of $20\text{k}\Omega$ (this peak-peak value = $2 \times$ the sinewave amplitude) at a frequency $f = 1$ cycle per 24 hours = $1/24 \text{ hrs}^{-1}$. Since resistance can never be negative for simple resistors, offset the sinewave by adding a constant $10\text{k}\Omega$ to it. Create an array representing time that counts every hour for seven full cycles = $7 \times 24 = 168 \text{ hrs}$.

NOTE: You are creating a resistance array that oscillates sinusoidally, but your input voltage *vp* will be a distorted sinewave due to the nonlinear effect of the voltage-divider equation transforming the resistance array to a voltage array.

Here is example code for a sinewave with 25mV amplitude at 250 Hz taken over four cycles:

```
f = 250; % 250 Hz temporal frequency
omega = 2*pi*f; % Angular frequency
t = linspace(0,1e-3,100); % 4 cycles*(1 sec/250 cycles), 100 pts
in = 25e-3*sin(omega*t); % This is an array the same size as t
```

- e) Use two subplots, one on top of the other, with the top subplot showing the inputs (*vp* and *vn*), and the bottom subplot showing the output. Include annotations and labels.

```
subplot(2,1,1) % Top subplot = inputs
plot(...)
title(...)
xlabel(...)
ylabel(...)
legend(% More than one waveform on a plot, legend has it
subplot(2,1,2) % Bottom subplot = output
plot(...)
title(...)
xlabel(...)
ylabel(...)
```